

Classic database MVCC versions rows by commit timestamp so readers never block writers. A **block executor** versions by **transaction index in a already-ordered block**. That is the MVCC you implement for parallel game-chain execution.

The serial story is: run $T_0, T_1, \dots, T_{n-1}$ in order against committed object state. MVCC lets workers run those txs at the same time while still answering every read with “the value the highest earlier writer produced.”

Version identity

A version is not “object v3.” It is:

$$\text{Version} = (\text{txn_index}, \text{incarnation})$$

- `txn_index` — position in the block (the preset serial order).
- `incarnation` — how many times that tx has been speculatively executed. First try is 0; each abort increments it.

Writes from incarnation k of T_i **replace** writes from incarnation $k - 1$ of the same T_i . Higher-index txs must never become visible to lower-index txs.

Core map (Aptos-shaped)

Production Block-STM is roughly:

Rust
<code>DashMap<K, BTreeMap<TxnIndex, CachePadded<WriteCell<V>>>></code>

- Outer map: one entry per **location** (object id, or `(object_id, field)`).
- Inner ordered map: writers of that location, keyed by `txn_index`.
- Cell: incarnation + value, or a marker `ESTIMATE`.

`DashMap` gives concurrent insert of new keys. The `BTreeMap` makes “latest writer with index $< j$ ” an $O(\log W)$ predecessor query. `CachePadded` reduces false sharing when many cores poke nearby cells.

For a game chain, K is `ObjectId` (and optionally a field). V is the object blob or a

• `AtomicU64`

cneap Arc<[u8]> .

Types

Rust

```
type TxnIndex = u32;
type Incarnation = u32;

#[derive(Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
struct Version {
    txn: TxnIndex,
    inc: Incarnation,
}

enum WriteEntry<V> {
    Value { inc: Incarnation, data: V },
    Estimate { inc: Incarnation }, // previous incarnation aborted; wait
}

struct WriteCell<V> {
    entry: parking_lot::RwLock<WriteEntry<V>>,
}

struct MvStore<K, V> {
    data: dashmap::DashMap<K, BTreeMap<TxnIndex, WriteCell<V>>>,
    // last write-set per txn, so abort can convert cells to Estimate
    last_writes: Vec<parking_lot::Mutex<Vec<K>>>,
}

enum ReadResult<V> {
    FromMv { version: Version, value: V },
    FromStorage { value: V }, // committed state before this block
    Estimate,                // must wait / reschedule
}
```

V should be Arc-like so a read copies a pointer, not a 4 KB NFT.

Read rule (READLAST)

When T_j reads key k :

1. Look in T_j 's **local write buffer** (read-your-writes).
2. Else look in the MV map for the greatest `txn_index` $i < j$ that wrote k .
3. If that cell is `Value` with incarnation c , return (i, c) and the bytes.
4. If it is `Estimate`, T_j has a **known dependency** on T_i — do not guess; yield to the scheduler.
5. If nobody earlier in the block wrote k , read **committed storage** (pre-block object version).

That is the whole visibility invariant: T_j never sees T_{j+1} , and it sees the latest *finished* earlier writer.

Rust

```
impl<K: Hash + Eq + Clone, V: Clone> MvStore<K, V> {
    fn read(&self, key: &K, reader: TxnIndex, storage: &dyn Storage<K, V>) ->
    ReadResult<V> {
        let Some(shard) = self.data.get(key) else {
            return ReadResult::FromStorage { value: storage.get(key) };
        };
        let pred = shard.range(..reader).next_back();
        match pred {
            Some((&txn, cell)) => match &*cell.entry.read() {
                WriteEntry::Estimate { .. } => ReadResult::Estimate,
                WriteEntry::Value { inc, data } => ReadResult::FromMv {
                    version: Version { txn, inc: *inc },
                    value: data.clone(),
                },
            },
            None => ReadResult::FromStorage { value: storage.get(key) },
        }
    }
}
```

Write rule

A write does **not** overwrite committed state. It inserts $(\text{txn_index} \rightarrow \text{cell})$ for that key.

- Same txn, new incarnation: overwrite the cell in place (same BTree key).
- First write from this txn: insert a new BTree entry.

- Record the key in `last_writes[txn]` for later abort/validate.

Write–write is not a conflict in this design. Both versions sit in the tree. The next reader picks the right predecessor. That is why Block-STM “avoids write–write conflicts.”

Rust

```
fn write(&self, key: K, writer: TxnIndex, inc: Incarnation, value: V) {
    let mut shard = self.data.entry(key.clone()).or_default();
    shard
        .entry(writer)
        .and_modify(|cell| {
            *cell.entry.write() = WriteEntry::Value { inc, data: value.clone() };
        })
        .or_insert_with(|| WriteCell {
            entry: parking_lot::RwLock::new(WriteEntry::Value { inc, data: value
        })),
        });
    self.last_writes[writer as usize].lock().push(key);
}
```

Execute an incarnation

Each worker runs the contract against a **view** that intercepts load/store:

Rust

```
struct ExecView<'a, K, V> {
    txn: TxnIndex,
    inc: Incarnation,
    store: &'a MvStore<K, V>,
    storage: &'a dyn Storage<K, V>,
    local: HashMap<K, V>,
    reads: Vec<(K, ReadDescriptor<V>>>,
}

enum ReadDescriptor<V> {
    Storage,
    Mv(Version),
}
```

On load: check `local` , else `store.read` , push a descriptor. On store: put in `local` only until the incarnation finishes, then flush `local` into the MV map as the write-set.

If any load returns `Estimate` , abort this attempt without publishing writes (or publish nothing new) and tell the scheduler “wait on txn i .”

Validate

Validation is **not** “re-run the contract.” It is: for every key in the recorded read-set, read the MV map again and check the version is still the one you saw.

Rust

```
fn validate(&self, txn: TxnIndex, reads: &[(K, ReadDescriptor<V>)], storage: &dyn
Storage<K, V>) -> bool {
    for (key, recorded) in reads {
        let now = self.read(key, txn, storage);
        match (recorded, now) {
            (ReadDescriptor::Storage, ReadResult::FromStorage { .. }) => {}
            (ReadDescriptor::Mv(v), ReadResult::FromMv { version, .. }) if version
== *v => {}
            _ => return false, // an earlier txn wrote (or rewrote) this key
        }
    }
    true
}
```

If T_i aborts and re-executes, every T_j with $j > i$ that read a key T_i writes must be re-validated, even if those T_j already validated once. Commit is only prefix-stable: you commit $T_0..T_k$ when that prefix has all validated and no earlier txn can still abort.

Abort and ESTIMATE

On failed validation:

1. Increment incarnation.
2. Take `last_writes[txn]` .
3. For each key, set the cell to `Estimate` **instead of deleting it**.
4. Re-execute. New writes overwrite `Estimate` with a real `Value` .

Why the marker: a later txn that already raced and read that key would otherwise see *stale*

...ing the incarnation later and that already, read and read history, would cause this to be stale storage and look valid. Estimate means “someone before you in serial order is about to write here; wait.” That is Block-STM’s dependency hint and the main abort-reduction trick.

Rust

```
fn mark_estimate(&self, txn: TxnIndex, inc: Incarnation) {
    let keys = std::mem::take(&mut *self.last_writes[txn as usize].lock());
    for key in keys {
        if let Some(mut shard) = self.data.get_mut(&key) {
            if let Some(cell) = shard.get(&txn) {
                *cell.entry.write() = WriteEntry::Estimate { inc };
            }
        }
    }
}
```

Never leave a torn Value from a doomed incarnation visible.

Commit and GC

When the prefix $T_0..T_k$ is final:

- Apply each tx’s last write-set to durable object storage **in order**.
- Drop MV entries with `txn_index ≤ k` (or drop the whole map when the block is done — Block-STM MV is **per-block, in-memory**).
- Object ids that were only estimated and never rewritten can be ignored.

Persistent object MVCC (Postgres-style version chains on disk) is a different layer: that is your committed store’s snapshot story for RPC/indexers, not the parallel executor.

Concurrency in Rust without fighting the checker

Problem	Pattern
Many cores, many keys	DashMap shards; do not hold a shard guard across VM execution
Ordered predecessor	BTreeMap inside the shard; keep write cardinality per key small
Cell updates	parking_lot::RwLock or arc_swap on the value; Arc<V> so reads are cheap

False sharing	CachePadded<WriteCell>
Per-txn metadata	Vec<Mutex<...>> indexed by txn , not a global map
Workers	Rayon pool or a custom collaborative scheduler (exec vs validate queues)
Determinism	no HashMap iteration in the VM; sort write-sets before apply

Do **not** use `unsafe` to alias object bytes. Publish new `Arc`s.

Object-centric wrinkle

On a game chain a “location” is usually the whole object. That coarsens conflicts (two field updates collide) but keeps the predecessor query trivial and matches owned vs shared objects.

Refinements:

- Split hot objects into fields (`(ObjectId, FieldId)`) for a marketplace pool.
- Owned-only txs can **skip MVCC**: greedy-commit into a private buffer, then splice into storage. No incarnation dance.
- Shared-object txs go through this map.

Mental walkthrough

Block: T_0 transfers sword A, T_1 transfers sword B, T_2 lists sword A on a shared market.

1. T_0 and T_1 execute together. Reads miss the MV map, hit storage. Writes land at indices 0 and 1.
2. T_2 reads A, predecessor is T_0 's new version. Lists it.
3. Validation: T_0 , T_1 match storage; T_2 still sees T_0 's version. Commit all three.
4. If T_0 had aborted after T_2 ran, A's cell becomes `Estimate` , T_2 validation fails, T_2 waits, then re-reads T_0 's new incarnation.

Same final state as serial $T_0 \rightarrow T_1 \rightarrow T_2$.

What this MVCC is not

- Not snapshot isolation with random commit timestamps. Order is the block.

- Not “readers never retry.” Higher txs retry when a lower tx changes their read-set.
- Not durable history. GC after the block.
- Not a substitute for object ownership. MVCC is how you run the **shared / residual-conflict** slice; the fast path should never enter this map.

Implement the map and the read/write/validate/estimate protocol first, with a single-threaded scheduler and tests that compare against a serial interpreter. Only then add the collaborative scheduler. Correctness of parallel execution lives almost entirely in this structure